

Helpdesk Email Channel

Frontend Integration Guide

Zoho-style email channel for the Obiyen helpdesk — Phases 1–5

Backend API reference & Nuxt integration notes

Scope: **insight-server** (Laravel) backend, complete & verified. Frontend (**insight-client**, Nuxt) not yet built — this document is the contract for building it.

Generated for the AWORK / Obiyen helpdesk module.

1. Overview

This document describes the backend API for the new helpdesk **Email Channel** — modeled on Zoho Desk's "Setting up your Email Channel". It covers everything delivered across Phases 1–5 plus the related helpdesk endpoints a frontend needs. All of it is server-side and verified; there is currently **no UI**.

How the channel works

- **Inbound:** customers email your real support address (e.g. `support@company.com`). You add a mail-forward rule to a generated *desk address*. All desk addresses land in one catch-all IMAP mailbox the system polls each minute; each message is routed by its recipient to the right department and becomes a ticket.
- **Outbound:** agents reply from inside the app. Replies are sent from a verified *From Address* (platform mailer, custom SMTP, or OAuth) with correct threading headers so customer replies stay on the same ticket.
- **Advanced:** org-level and per-department toggles (auto-Cc, agent name, reply-to contact, original sender, auto-private, thread-by-ticket-id).
- **Bounces:** delivery failures are detected, classified (hard/soft), and blocked; the app refuses to send to blocked addresses.

Conventions

Field	Type / values	Notes
Base URL	string	Authenticated endpoints are under <code>/api/user/helpdesk</code> . Public ones under <code>/api/public/helpdesk</code> .
Auth	Bearer	Sanctum token in the <code>Authorization: Bearer <token></code> header (same as the rest of the user API). Public endpoints need no auth.
Tenancy	implicit	Every authenticated request is scoped to the caller's <code>client_id</code> . Never send <code>client_id</code> from the frontend.
IDs	uuid	Resources are addressed by uuid (departments use <code>slug</code> in some routes). Internal numeric ids are never exposed.
Secrets	write-only	Passwords / tokens are never returned. Resources expose booleans like <code>password_set</code> / <code>oauth_connected</code> instead.

All request bodies are JSON. All list endpoints return Laravel API-resource collections under a `data` key unless noted.

2. Phase 1 — Inbound Email Channel

Two resources: the **IMAP catch-all connection** (one per tenant) and the **Support Email Addresses** (aliases that map an external address to a generated desk address and a department).

2.1 IMAP catch-all connection

Method	Endpoint	Purpose
GET	/api/user/helpdesk/imap-connection	Current mailbox config (or null)
PUT	/api/user/helpdesk/imap-connection	Create / update the mailbox (one per tenant)
DELETE	/api/user/helpdesk/imap-connection	Remove the mailbox
POST	/api/user/helpdesk/imap-connection/test	Test credentials (200 ok / 422 fail)
POST	/api/user/helpdesk/imap-connection/toggle	Master fetch on/off

PUT body

Field	Type / values	Notes
host	string, required	IMAP host, e.g. imap.gmail.com
port	int	Default 993
username	string, required	Mailbox login
password	string	Write-only; omit on edit to keep the stored one
encryption	ssl tls none	Default ssl
validate_cert	bool	Default true
folder	string	Default INBOX
is_active	bool	Master fetch toggle

Response shape (data)

```
{
  "uuid": "_", "host": "imap.gmail.com", "port": 993,
  "username": "box@company.com", "encryption": "ssl",
  "validate_cert": true, "folder": "INBOX", "is_active": true,
  "password_set": true, // never the password itself
  "last_polled_at": "2026-05-28T...", "last_error": null,
  "created_at": "2026-05-28T..."
}
```

Toggle body: { "active": true|false }. **Test** returns { "ok": bool, "error": string|null }.

2.2 Support Email Addresses (aliases)

Method	Endpoint	Purpose
GET	/api/user/helpdesk/support-addresses	List aliases
POST	/api/user/helpdesk/support-addresses	Create alias (409 if external exists)
PATCH	/api/user/helpdesk/support-addresses/{uuid}	Edit friendly name / map department
DELETE	/api/user/helpdesk/support-addresses/{uuid}	Remove alias
POST	/api/user/helpdesk/support-addresses/{uuid}/fetch	Start/stop fetch for this alias

POST body

Field	Type / values	Notes
external_address	email, required	Address customers email / forward FROM (support@company.com)
alias	string	Desired local-part of the desk address; defaults to the external local-part
friendly_name	string	Display name
department_uuid	uuid	Department this alias routes to
fetch_enabled	bool	Default true

Response shape

```
{
  "uuid": "_",
  "external_address": "support@company.com",
  "desk_address": "support-7@desk.obiyen.com", // generated; forward TO this
  "friendly_name": "Support",
  "fetch_enabled": true,
  "is_verified": false, // flips true on first mail received
  "verified_at": null,
  "department": { "uuid": "_", "slug": "support", "name": "Support" },
  "created_at": "2026-05-28T_"
}
```

Fetch toggle body: { "enabled": true|false }. **UI flow:** create the alias → show the customer the desk_address to forward to → is_verified turns true automatically once the first forwarded mail arrives.

Behind the scenes: a scheduled command helpdesk:poll-imap runs every minute, fetches new mail, routes by recipient → alias → department, and creates/threads tickets. No frontend action needed.

3. Phase 2 — From Address management

From Addresses are the identities agents reply from, per department. Each must be sendable before use: the **app** method requires email verification (+ SPF); **smtp** and **oauth** carry their own transport.

Method	Endpoint	Purpose
GET	/api/user/helpdesk/from-addresses	List From Addresses
POST	/api/user/helpdesk/from-addresses	Add (409 if duplicate)
PATCH	/api/user/helpdesk/from-addresses/{uuid}	Update
DELETE	/api/user/helpdesk/from-addresses/{uuid}	Remove
POST	/api/user/helpdesk/from-addresses/{uuid}/resend-verification	Resend verification email (app method)
POST	/api/user/helpdesk/from-addresses/{uuid}/set-default	Make default for its department
GET	/api/public/helpdesk/from-address/verify/{token}	PUBLIC — verification link target

POST body

Field	Type / values	Notes
address	email, required	The From address
friendly_name	string	Display name on the From line
department_uuid	uuid	Owning department
send_method	app smtp oauth	Default app
is_default / is_active	bool	Default flag / enabled flag
smtp_host, smtp_port, smtp_username, smtp_password, smtp_encryption	required_if smtp	Custom SMTP transport (creds write-only)
oauth_provider	google microsoft outlook	required_if oauth
oauth_refresh_token	string	Stored encrypted; presence ⇒ connected

Response shape

```
{
  "uuid": "...", "address": "reply@company.com", "friendly_name": "Support",
  "send_method": "app",
  "is_default": true, "is_active": true,
  "verification_status": "pending", // pending | verified | failed
  "verified_at": null,
  "is_sendable": false, // true once usable
  "department": { "uuid": "...", "slug": "support", "name": "Support" },
  "smtp_host": null, "smtp_port": null, "smtp_username": null,
  "smtp_encryption": null, "smtp_password_set": false,
  "oauth_provider": null, "oauth_connected": false,
  "spf_record": "v=spf1 mx include:app.obiyen.com ~all", // only for app method
  "created_at": "2026-05-28T..."
}
```

UI flow (app method): create → status is pending and a verification email is sent → show the spf_record to add to DNS → the recipient clicks the emailed link (/api/public/helpdesk/from-address/verify/{token}) → status becomes verified and

`is_sendable` true. Offer a **Resend** button.

smtp method is verified immediately once credentials are saved (no SPF needed). oauth becomes verified once a refresh token is connected. If a send later fails authentication, status flips to `failed` — surface a **Re-authenticate** affordance.

4. Phase 3 — Outbound transports (sending)

This phase is entirely backend: when an agent reply is queued, the system picks the right transport for the ticket's department — a sendable From Address (app / SMTP / OAuth-XOAUTH2, with token refresh) → legacy department provider → platform default. **No dedicated frontend endpoints**; the frontend influence is only through Phase 2 (which From Address is default/sendable) and the reply composer (Section 7).

- OAuth credentials are configured server-side under `config/services.helpdesk_oauth.{google,microsoft}` (env-driven).
- Threading headers (Message-ID / References / In-Reply-To) and the `Re: ... [#AW-YYYY-NNNN]` subject are added automatically.
- On an auth-type send failure the From Address is marked failed for re-authentication.

5. Phase 4 — Advanced configuration

Org-level defaults with optional per-department overrides. A null value at department scope inherits the org value, which inherits hardcoded defaults.

Method	Endpoint	Purpose
GET	<code>/api/user/helpdesk/email-settings</code>	Org-level settings (resolved + own)
PATCH	<code>/api/user/helpdesk/email-settings</code>	Update org-level
GET	<code>/api/user/helpdesk/departments/{slug}/email-settings</code>	Per-department (resolved + own)
PATCH	<code>/api/user/helpdesk/departments/{slug}/email-settings</code>	Update per-department override

Settings fields (all nullable at department scope = inherit)

Field	Type / values	Notes
<code>auto_cc_support_mailbox</code>	bool	Cc the support mailbox on every reply
<code>create_contact_with_reply_to</code>	bool	Use the Reply-To header for the contact, not From
<code>show_agent_name</code>	bool	Put the agent's name on the From line
<code>forwarded_use_original_sender</code>	bool	Agent-forwarded mail with <code>#original_sender</code> → use the real customer
<code>auto_private_on_recipient_mismatch</code>	bool	Reply that excludes the requestor becomes a private note
<code>thread_using_ticket_id</code>	bool	Thread inbound mail by <code>###NNN##</code> in the subject
<code>thread_ticket_id_scope</code>	agents participants	Who may thread by ticket id (anti-spam)

GET response shape

```
{
  "data": {
    "resolved": { ...effective values the pipeline uses... },
    "own":      { ...this scope's stored values; null = inherit... },
    "scope":    "org" // or "department"
  }
}
```

UI tip: render toggles from own (tri-state: on / off / inherit) and show the effective value from resolved as a hint.

6. Phase 5 — Bounce handling

Delivery failures are detected from inbound DSNs, classified, and tracked. Hard bounces block immediately; 5 soft bounces convert to a permanent block. The reply pipeline refuses to send to blocked addresses.

Method	Endpoint	Purpose
GET	/api/user/helpdesk/bounces	List bounces (optional ?blocked=true false)
DELETE	/api/user/helpdesk/bounces/{uuid}	Unblock (remove from block list)

Response shape

```
{
  "uuid": "_", "email": "dead@client.com",
  "type": "hard", // hard | soft
  "reason": "550 mailbox unavailable",
  "soft_count": 0,
  "is_blocked": true,
  "blocked_at": "2026-05-28T_", "last_bounced_at": "2026-05-28T_",
  "ticket_uuid": "_" // originating ticket if matched
}
```

UI tip: a "Suppressed addresses" table with a filter for blocked, each row showing type/reason and an **Unblock** action (the DELETE endpoint). Recording of bounces is automatic (inbound pipeline).

7. Reply composer (recipients-aware)

The agent reply endpoint accepts an optional recipient list and Cc, which drive the auto-private behavior.

Method	Endpoint	Purpose
GET	/api/user/helpdesk/tickets/{uuid}/messages	List the ticket conversation
POST	/api/user/helpdesk/tickets/{uuid}/messages	Send a reply or internal note

POST body

Field	Type / values	Notes
body_html	string, required	Reply/note body (max ~100KB)
is_internal_note	bool	true = private note (not emailed)
recipients	array<email>	To recipients; omit ⇒ defaults to the ticket requestor
cc	array<email>	Cc recipients

- If recipients is provided and EXCLUDES the requestor, and auto_private_on_recipient_mismatch is on, the message is stored as a private note (no email sent).
- Blocked (bounced) addresses are dropped from To/Cc automatically; if none remain, no email is sent.
- auto_cc_support_mailbox adds the support mailbox to Cc automatically.

UI: add a To/Cc selector to the composer, defaulting To to the requestor, plus a "reply privately" affordance.

8. Related helpdesk endpoints (reference)

These pre-existing endpoints are what the email-channel screens sit alongside. Included so the integration is complete.

Method	Endpoint	Purpose
GET	/api/user/helpdesk/stats	Sidebar counts (open/pending/...)
GET	/api/user/helpdesk/analytics	Analytics
GET	/api/user/helpdesk/departments	Departments (+ counts)
POST/PATCH/DELETE	/api/user/helpdesk/departments/{slug}	Department CRUD
GET	/api/user/helpdesk/departments/{uuid}	Department detail
GET/POST	/api/user/helpdesk/contacts	Contacts list / create
GET	/api/user/helpdesk/contacts/{uuid}/tickets	Contact detail / their tickets
GET/POST/PATCH/DELETE	/api/user/helpdesk/tags/{uuid}	Tags CRUD
GET	/api/user/helpdesk/agents	Agents
GET/POST/PATCH/DELETE	/api/user/helpdesk/tickets/{uuid}	Ticket CRUD
POST	/api/user/helpdesk/tickets/{uuid}/ai/{suggest-reply summarize suggest-tags}	AI helpers
GET/POST/PATCH/DELETE	/api/user/helpdesk/canned-responses/{uuid}	Canned responses
GET/POST/PATCH/DELETE	/api/user/helpdesk/kb/{categories articles}/{uuid}	Knowledge base
GET/POST/PATCH/DELETE	/api/user/helpdesk/companies/{slug uuid}	Companies
various	/api/user/helpdesk/departments/{slug}/inbox ...	Legacy Gmail-connect inbox (OAuth)
POST	/api/public/helpdesk/webhooks/email/inbound	PUBLIC inbound webhook (provider push)

9. Suggested frontend screens

A "Setup → Channels → Email" area, mirroring Zoho:

- **Mailbox** — IMAP connection form (host/port/user/password/encryption/folder), Test & Save, active toggle, last-poll/error status.

- **Support Addresses** — alias table: external ↔ desk address, department mapping, verified badge, fetch toggle, copy-desk-address button + forwarding instructions.
- **From Addresses** — table with verification status / sendable badge, add modal (app/SMTP/OAuth tabs), SPF record display, Resend / Set-default / Re-authenticate actions.
- **Advanced** — the seven toggles (tri-state at department scope), org vs per-department tabs.
- **Suppressed (Bounces)** — bounce table with type/reason and Unblock.
- **Ticket reply composer** — To/Cc selector + private-reply toggle (Section 7).

API service pattern (Nuxt)

Add a `HelpdeskEmailChannelService` extending `BaseAPIService` (Bearer token handled by the base). Example:

```
// components/api/user/HelpdeskEmailChannelService.ts
class HelpdeskEmailChannelService extends BaseAPIService {
  getImapConnection() { return this.request("/user/helpdesk/imap-connection", "GET") }
  saveImapConnection(body) { return this.request("/user/helpdesk/imap-connection", "PUT", body) }
  testImapConnection() { return this.request("/user/helpdesk/imap-connection/test", "POST") }
  listSupportAddresses() { return this.request("/user/helpdesk/support-addresses", "GET") }
  createSupportAddress(b) { return this.request("/user/helpdesk/support-addresses", "POST", b) }
  listFromAddresses() { return this.request("/user/helpdesk/from-addresses", "GET") }
  getEmailSettings() { return this.request("/user/helpdesk/mail-settings", "GET") }
  listBounces(blocked) { return this.request(`/user/helpdesk/bounces?blocked=${blocked}`, "GET") }
  sendReply(uuid, body) { return this.request(`/user/helpdesk/tickets/${uuid}/messages`, "POST", body) }
}
```

10. Operational prerequisites (go-live)

These are infra/config steps outside the frontend, required before the channel works end to end:

- Set `HELPDESK_DESK_DOMAIN` and point its **MX** at the catch-all mailbox host.
- Save the IMAP mailbox via `PUT /imap-connection`; ensure a queue worker and the scheduler (`helpdesk:poll-imap`, every minute) are running.
- For OAuth sending: real Google / Microsoft client credentials in env + a connected refresh token.
- For app-method From Addresses: add the returned `spf_record` to the sending domain's DNS.

This guide reflects the backend as implemented and verified in insight-server (Phases 1–5). Endpoints, request bodies, and response shapes are taken directly from the routes, form requests, and API resources.